

Machine Learning Assignment 2

TumorScope ML Classification Studio

M.Tech (AIML / DSE) | Work Integrated Learning Programmes

Student Name	REPLACE WITH YOUR NAME
Student ID	2025ad05067
Dataset	Breast Cancer Wisconsin (Diagnostic)
Submission Date	REPLACE WITH FINAL SUBMISSION DATE

Six classification pipelines | 30 features | 569 instances | Interactive Streamlit deployment

1. Mandatory Submission Links

The final submission PDF must contain clickable links to the complete GitHub repository and the deployed Streamlit application.

GitHub Repository	REPLACE WITH YOUR GITHUB REPOSITORY URL
Live Streamlit App	REPLACE WITH YOUR LIVE STREAMLIT URL
Entry Point	app.py
Test Data	test_data.csv

Action required: Replace both link placeholders after the repository and app are live. Rebuild the report before submission.

2. Problem Statement

The objective is to implement multiple classification models on one public dataset, evaluate every model with Accuracy, AUC, Precision, Recall, F1 Score, and Matthews Correlation Coefficient, and demonstrate the saved models through an interactive Streamlit web application.

Model-count interpretation: The brief names five mandatory models but also refers to six models. This project includes every named model and adds a Support Vector Machine as the sixth classifier.

3. Dataset Description

The Breast Cancer Wisconsin (Diagnostic) dataset is used because it satisfies the minimum size constraints and supports a clear binary classification task. The predictors are numeric measurements derived from digitized images of breast-mass cell nuclei.

Dataset Field	Value
Dataset	Breast Cancer Wisconsin (Diagnostic)
Repository	UCI Machine Learning Repository
Instances	569
Predictor Features	30
Task	Binary classification
Training Rows	455
Test Rows	114
Split	Stratified 80:20 split

Dataset Field	Value
Random State	42
Target Mapping	0 = Benign, 1 = Malignant
Positive Class	Malignant (1)

Dataset source: <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>

Feature organization

The 30 predictors contain ten base measurements represented as mean values, standard-error values, and worst/extreme values. The full feature dictionary is included in data/feature_dictionary.csv.

4. Modeling Methodology

- Normalize all predictor names to lowercase snake_case.
- Remap the target to 0 = Benign and 1 = Malignant.
- Create one stratified 80:20 train-test split with random_state=42.
- Use median imputation in every saved pipeline so uploaded CSV files can contain missing numeric cells.
- Standardize features for distance-based, linear, probabilistic, and kernel models.
- Train all six classifiers on the same training rows and evaluate them on the same 114 test rows.
- Save each fitted pipeline as a joblib artifact for Streamlit deployment.

Implemented models

Model	Preprocessing	Key Configuration
Logistic Regression	Median imputation + standardization	Balanced class weights; max_iter=5000
Decision Tree	Median imputation	max_depth=5; min_samples_leaf=4; balanced weights
K-Nearest Neighbors	Median imputation + standardization	k=7; distance weighting
Gaussian Naive Bayes	Median imputation + standardization	var_smoothing=1e-9
Random Forest	Median imputation	400 trees; min_samples_leaf=2; balanced weights
Support Vector Machine	Median imputation + standardization	RBF kernel; C=2.0; probability=True; balanced weights

5. Evaluation Results

All metrics use Malignant (1) as the positive class. AUC is calculated from the model probability or decision score rather than the final thresholded class.

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9737	0.9954	0.9756	0.9524	0.9639	0.9433

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Decision Tree	0.9035	0.8980	0.8974	0.8333	0.8642	0.7908
K-Nearest Neighbors	0.9561	0.9835	0.9744	0.9048	0.9383	0.9058
Gaussian Naive Bayes	0.9211	0.9891	0.9231	0.8571	0.8889	0.8292
Random Forest	0.9737	0.9974	1.0000	0.9286	0.9630	0.9442
Support Vector Machine	0.9825	0.9950	1.0000	0.9524	0.9756	0.9626

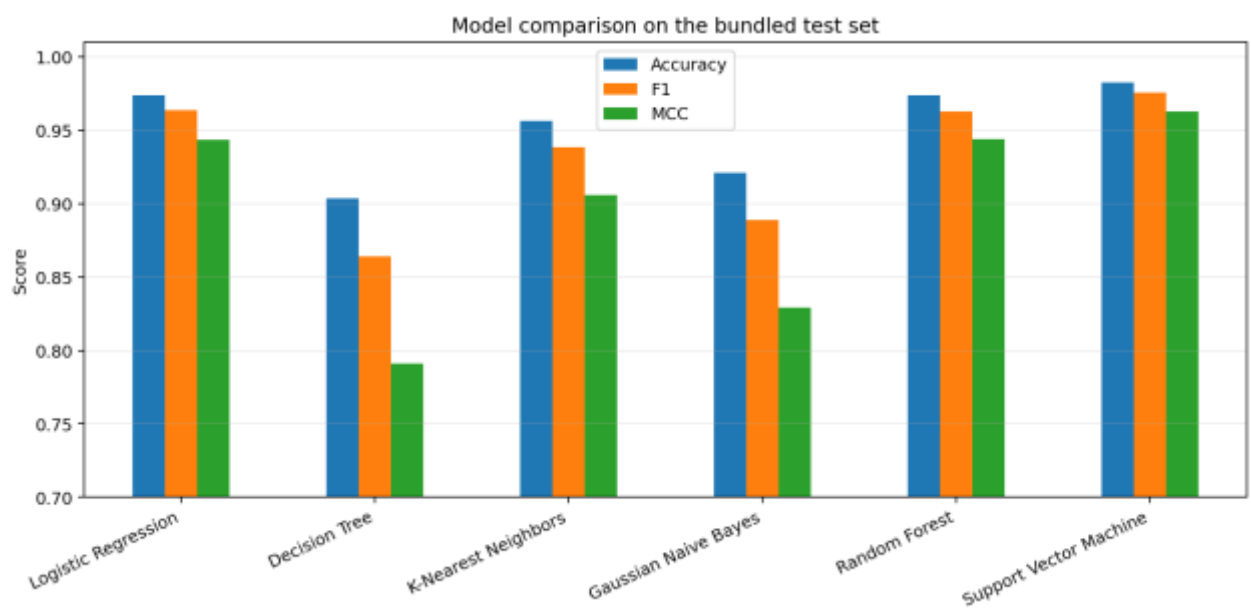


Figure 1. Accuracy, F1, and MCC comparison

Winner confusion matrix

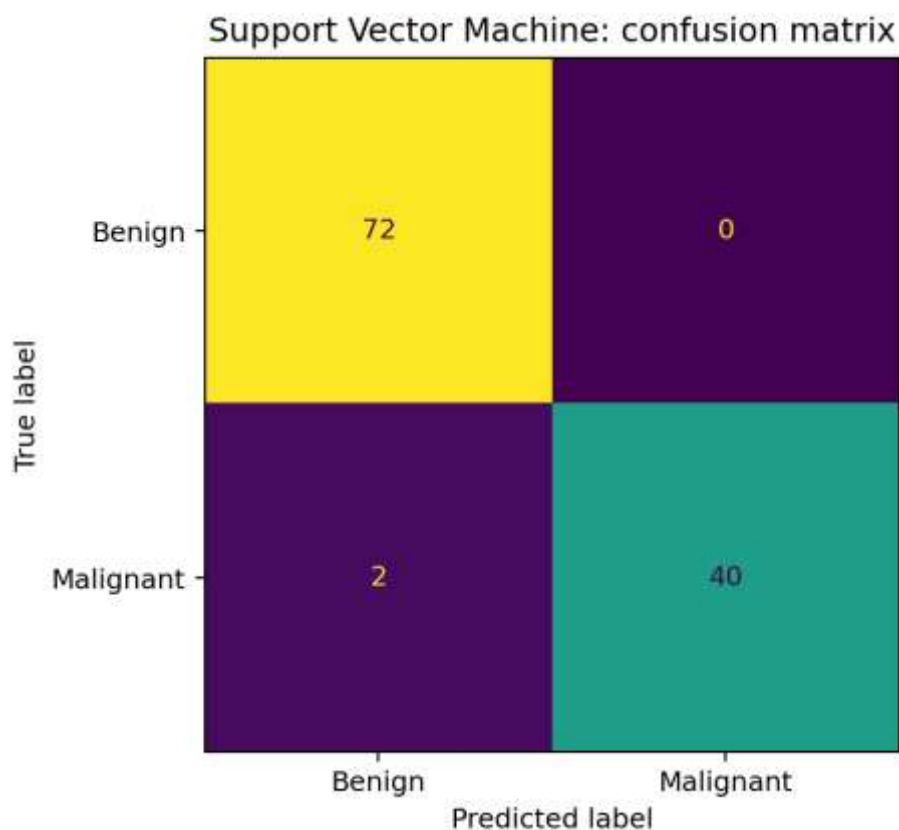


Figure 2. Support Vector Machine predictions on the bundled test set

6. Model Performance Observations

ML Model Name	Observation about model performance
Logistic Regression	Strong linear baseline after feature standardization. It achieved 0.9737 accuracy, 0.9524 recall, and 0.9433 MCC.
Decision Tree	Lowest overall result on this split. The lower recall and MCC indicate that one depth-controlled tree generalized less effectively than the scaled and ensemble models.
K-Nearest Neighbors	Good precision after scaling, with a strong MCC. It missed more malignant cases than Logistic Regression and SVM, which reduced recall and F1.
Gaussian Naive Bayes	Very high AUC but lower thresholded recall and F1. Its probability ranking was strong, while the conditional-independence assumption limited the final class decisions.
Random Forest	Best AUC at 0.9974 and perfect malignant-class precision on this test split. It produced no false-positive malignant predictions but missed three malignant rows.
Support Vector Machine	Best overall balance under the MCC-first ranking. It achieved the highest accuracy, F1, and MCC, together with perfect precision and high recall.

ML Model Name	Observation about model performance
Overall Winner	Support Vector Machine. The ranking rule uses MCC first, then F1, AUC, and Accuracy. Random Forest remains the AUC winner.

7. Streamlit Application and Repository

Required application features

- CSV upload control with bundled test_data.csv as the default evaluation file.
- Model-selection dropdown covering all six saved classifiers.
- Metric cards for Accuracy, AUC, Precision, Recall, F1, and MCC.
- Confusion matrix and classification report for labeled test data.
- All-model comparison table and performance chart.
- Row-level class predictions and malignant-class probabilities.
- Prediction CSV download control.
- Feature validation, target conversion, and missing-value handling.

Repository structure

```
project-folder/
|-- app.py
|-- requirements.txt
|-- README.md
|-- test_data.csv
|-- student_config.json
|-- data/
|-- model/
|   |-- train_models.py
|   |-- model_registry.py
|   `-- artifacts/ (six saved classifiers and metric files)
|-- utils/
|-- tests/
|-- tools/build_submission_report.py
`-- report/submission_report.docx and submission_report.pdf
```

Local execution

```
python -m pip install -r requirements.txt
python -m model.train_models
streamlit run app.py
```

Expected result: a browser opens with the upload control, model dropdown, six metric cards, confusion matrix, classification report, comparison table, and prediction download button.

8. BITS Virtual Lab Execution Evidence

The assignment requires one screenshot showing successful execution on BITS Virtual Lab. The screenshot must be captured by the student in the required environment.

INSERT GENUINE BITS VIRTUAL LAB SCREENSHOT HERE

Save it as report/bits_virtual_lab_screenshot.png and rebuild the report.

9. Conclusion

The project completes the full machine learning workflow: dataset preparation, consistent model training, multi-metric evaluation, artifact persistence, interactive visualization, and deployment packaging. On the bundled test split, Support Vector Machine is the overall winner under the stated MCC-first ranking, while Random Forest achieves the highest AUC.

Important limitation: This is an educational classification demonstration and must not be used as a medical diagnostic system.

10. Final Submission Checklist

- ☐ GitHub repository link opens and contains complete source code.
- ☐ requirements.txt, README.md, and test_data.csv are present in the repository root.
- ☐ All six model artifacts are committed under model/artifacts/.
- ☐ Live Streamlit app opens in a private/incognito browser window.
- ☐ CSV upload, model selection, metrics, confusion matrix, and classification report work.
- ☐ README content is represented in this PDF.
- ☐ One genuine BITS Virtual Lab execution screenshot is included.
- ☐ Student name, BITS ID, GitHub URL, and Streamlit URL are finalized.

Appendix A. Reproducibility Notes

Run `python -m model.train_models` to regenerate the train/test CSV files, all six model artifacts, metrics.csv, and metadata.json. Run `pytest -q` after installing requirements-dev.txt; the expected result is 6 passed.

The included metrics are tied to the supplied dataset, the stratified split, `random_state=42`, and the recorded model settings. Changing any of these items can change the comparison table and the overall winner.